

# 软件工程

Software Engineering

MS Roslyn

2026 年 10 月 8 日

## 摘要

本教程系统讲解软件工程的核心方法与建模技术：从软件过程模型切入，涵盖需求工程、结构化分析与设计、面向对象与 UML 建模、数据建模与 ER、设计原则与模式、软件测试，以及项目管理与质量度量。教程强调「过程 + 建模 + 质量」三条主线，把抽象的方法论落到可操作的图表与度量上，帮助读者掌握从需求到交付的完整工程实践。

# 目录

|            |                         |           |
|------------|-------------------------|-----------|
| <b>第一章</b> | <b>软件工程概述与过程模型</b>      | <b>4</b>  |
| 1.1        | 软件与软件危机 . . . . .       | 5         |
| 1.2        | 软件过程模型 . . . . .        | 5         |
| 1.3        | 软件生命周期 . . . . .        | 5         |
| 1.4        | 敏捷与 Scrum . . . . .     | 5         |
| <b>第二章</b> | <b>需求工程</b>             | <b>6</b>  |
| 2.1        | 需求获取方法 . . . . .        | 7         |
| 2.2        | 需求管理 . . . . .          | 7         |
| 2.3        | 需求分类与规格说明 . . . . .     | 7         |
| 2.4        | 用例建模 . . . . .          | 8         |
| <b>第三章</b> | <b>结构化分析与设计</b>         | <b>9</b>  |
| 3.1        | 数据流图 (DFD) . . . . .    | 10        |
| 3.1.1      | 四种基本元素 . . . . .        | 10        |
| 3.1.2      | 分层 DFD . . . . .        | 10        |
| 3.1.3      | 平衡原则 . . . . .          | 10        |
| 3.2        | 数据字典 . . . . .          | 11        |
| 3.3        | 模块化设计: 耦合与内聚 . . . . .  | 11        |
| 3.3.1      | 耦合 (Coupling) . . . . . | 11        |
| 3.3.2      | 内聚 (Cohesion) . . . . . | 12        |
| <b>第四章</b> | <b>面向对象与 UML</b>        | <b>13</b> |
| 4.1        | 类图 . . . . .            | 14        |
| 4.1.1      | 类与成员 . . . . .          | 14        |
| 4.1.2      | 类之间的关系 . . . . .        | 15        |
| 4.1.3      | 多重性 . . . . .           | 15        |
| 4.1.4      | 类图示例 . . . . .          | 16        |
| 4.2        | 用例图 . . . . .           | 17        |
| 4.2.1      | 要素与关系 . . . . .         | 17        |
| 4.3        | 时序图 . . . . .           | 18        |
| 4.3.1      | 要素 . . . . .            | 18        |
| 4.4        | 状态图与活动图 . . . . .       | 19        |
| 4.4.1      | 状态图 . . . . .           | 19        |

|            |                           |           |
|------------|---------------------------|-----------|
| 4.4.2      | 活动图 . . . . .             | 20        |
| <b>第五章</b> | <b>数据建模与 ER</b>           | <b>21</b> |
| 5.1        | 实体、属性与键 . . . . .         | 22        |
| 5.2        | 联系与基数 . . . . .           | 22        |
| 5.3        | ER 到关系模式的转换 . . . . .     | 23        |
| 5.4        | 函数依赖与范式 . . . . .         | 24        |
| <b>第六章</b> | <b>设计原则与模式</b>            | <b>25</b> |
| 6.1        | SOLID 五原则 . . . . .       | 26        |
| 6.2        | 设计模式 . . . . .            | 26        |
| 6.2.1      | 模式分类 . . . . .            | 26        |
| 6.2.2      | 代表模式 . . . . .            | 27        |
| 6.3        | 软件架构 . . . . .            | 28        |
| <b>第七章</b> | <b>软件测试</b>               | <b>29</b> |
| 7.1        | 测试层次与 V 模型 . . . . .      | 30        |
| 7.2        | 黑盒测试与白盒测试 . . . . .       | 31        |
| 7.2.1      | 黑盒测试 . . . . .            | 31        |
| 7.2.2      | 白盒测试 . . . . .            | 31        |
| 7.3        | 测试用例设计与回归测试 . . . . .     | 32        |
| <b>第八章</b> | <b>项目管理与质量</b>            | <b>33</b> |
| 8.1        | 规模与成本估算 . . . . .         | 34        |
| 8.1.1      | 代码行 LOC 与功能点 FP . . . . . | 34        |
| 8.1.2      | COCOMO 模型 . . . . .       | 34        |
| 8.2        | 进度计划 . . . . .            | 35        |
| 8.2.1      | 甘特图 . . . . .             | 35        |
| 8.2.2      | PERT 与关键路径法 CPM . . . . . | 35        |
| 8.3        | 风险管理 . . . . .            | 36        |
| 8.3.1      | 风险识别与评估 . . . . .         | 36        |
| 8.3.2      | 风险应对策略 . . . . .          | 37        |
| 8.4        | 软件质量 . . . . .            | 37        |
| 8.4.1      | ISO 质量模型 . . . . .        | 37        |
| 8.4.2      | 质量度量 . . . . .            | 38        |

# 第一章 软件工程概述与过程模型

软件工程用「工程化」的方法控制软件的成本、进度与质量。本章建立「软件危机-过程模型-生命周期-敏捷」四块基石。

## 1.1 软件与软件危机

- **软件**：程序 + 数据 + 文档。它是逻辑产品，无物理磨损，但会因变更而「退化」。
- **软件危机**：需求不清、进度失控、成本超支、质量低、维护难。
- **软件工程**：将系统化、规范化、可度量的工程方法应用于软件的开发、运行与维护。

## 1.2 软件过程模型

表 1.1: 常见过程模型对比

| 模型             | 特点             | 适用       |
|----------------|----------------|----------|
| 瀑布 (Waterfall) | 阶段线性、文档驱动      | 需求稳定、合同型 |
| 迭代/增量          | 分轮交付，逐步完善      | 中大型、需求渐进 |
| 螺旋 (Spiral)    | 迭代 + 风险分析      | 高风险大型项目  |
| 原型 (Prototype) | 先做可运行原型澄清需求    | 需求模糊     |
| 敏捷 (Agile)     | 短迭代、响应变化、可工作软件 | 需求多变、小团队 |

## 1.3 软件生命周期



图 1.1: 软件生命周期的主要阶段

维护成本常占生命周期总成本的 60% ~ 80%；需求与设计缺陷的修复代价随阶段指数增长：

$$C_{\text{fix}}(\text{阶段}) \propto e^{k \cdot \text{阶段序号}} \tag{1.1}$$

## 1.4 敏捷与 Scrum

- **敏捷宣言**：个体与互动、可工作软件、客户合作、响应变化高于流程与工具、详尽文档、合同谈判、遵循计划。
- **Scrum**：Sprint（1-4 周）、产品待办列表 (Product Backlog)、冲刺待办列表、每日站会、评审与回顾。
- **XP**：结对编程、测试驱动开发 (TDD)、持续集成、小步发布。

表 1.2: Scrum 三种角色与三类工件

| 角色           | 工件             |
|--------------|----------------|
| 产品负责人 (PO)   | 产品待办列表         |
| Scrum Master | 冲刺待办列表         |
| 开发团队         | 增量 (Increment) |

## 第二章 需求工程

需求工程把「用户想要什么」转化为「可验证、可追溯、可管理的规格」，是决定项目成败的第一道关口。

## 2.1 需求获取方法

- **需求**：用户对目标系统的期望与约束，分为功能需求与非功能需求；它是设计、实现与验收的共同基准。
- **需求工程**：获取 (elicitation) → 分析 (analysis) → 规格说明 (specification) → 验证 (validation) → 管理 (management) 的迭代过程。
- **干系人 (Stakeholder)**：所有受系统影响或能影响系统的人与组织，是需求获取的信息来源。

表 2.1: 常用需求获取方法

| 方法                       | 做法            | 适用与要点               |
|--------------------------|---------------|---------------------|
| 访谈 (Interview)           | 与干系人一对一或小组交流  | 深入挖掘隐含需求；成本高、依赖访谈技巧 |
| 问卷 (Questionnaire)       | 结构化问题批量收集     | 覆盖广、易统计；难以追问细节      |
| 观察 (Observation)         | 到现场观察真实工作流    | 发现用户未说出口的需求；耗时、可能干扰 |
| 原型 (Prototype)           | 用可运行或低保真原型试错  | 澄清模糊需求；易被误当成最终产品    |
| 文档分析 (Document Analysis) | 分析现有表单、手册、旧系统 | 快速获取业务规则；文档可能已过时    |

## 2.2 需求管理

- **基线 (Baseline)**：需求通过评审后冻结，作为后续变更与验收的基准。
- **变更控制 (Change Control)**：变更申请 (CR) → 影响分析 → 批准或拒绝 → 更新基线，避免需求无序蔓延 (scope creep)。
- **追溯矩阵 (Traceability Matrix)**：在需求、设计、代码与测试用例之间建立双向链接，保证覆盖与影响分析。

## 2.3 需求分类与规格说明

表 2.2: 功能需求与非功能需求对比

| 维度   | 功能需求         | 非功能需求           |
|------|--------------|-----------------|
| 关注点  | 系统「做什么」      | 系统「做得怎样」        |
| 表述   | 输入 → 处理 → 输出 | 可度量的质量指标        |
| 示例   | 用户可下单、支付     | 响应 < 2s、并发 1000 |
| 验证方式 | 功能测试、验收测试    | 性能/安全/压力测试      |
| 失败后果 | 功能缺失或错误      | 系统不可用或不达标       |

- **性能 (Performance)**：响应时间、吞吐量、并发用户数、资源占用。
- **安全 (Security)**：认证、授权、加密、审计、抗攻击能力。

- **可用性 (Usability):** 易学习、易操作、可访问、出错可恢复（注意与 availability「可靠性/可用性」区分）。
- **可维护性 (Maintainability):** 可修改、可测试、可复用、低耦合高内聚。

需求规格说明 (SRS) 应满足以下特征：

- **正确 (Correct):** 真实反映干系人的需求。
- **无歧义 (Unambiguous):** 只有一种解释。
- **完整 (Complete):** 涵盖所有功能、约束与边界。
- **一致 (Consistent):** 条目之间无冲突、无矛盾。
- **可验证 (Verifiable):** 能用测试或度量客观判定是否满足。

## 2.4 用例建模

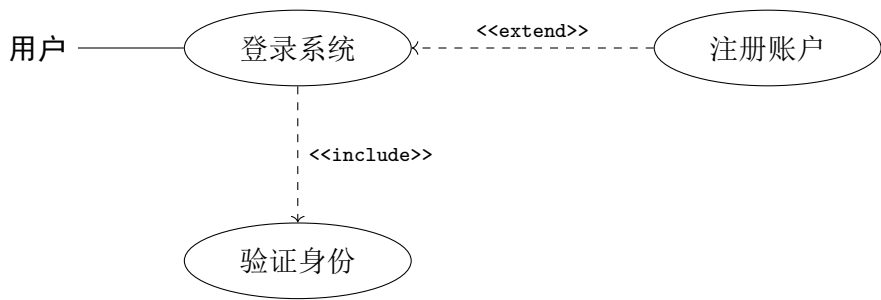


图 2.1: 用例图要素：参与者、用例、include 与 extend 关系

- **参与者 (Actor):** 与系统交互的角色，可以是人或外部系统。
- **用例 (Use Case):** 系统向参与者提供的一个完整、有价值的功能单元。
- **include:** 基础用例必然执行被包含用例（公共步骤抽取），箭头由基础用例指向被包含用例。
- **extend:** 扩展用例在特定条件下插入基础用例，基础用例不依赖扩展。
- **泛化 (Generalization):** 参与者或用例之间的继承 (is-a) 关系。

表 2.3: 用例描述模板

| 条目   | 内容              |
|------|-----------------|
| 用例名称 | 动词短语，如「提交订单」    |
| 参与者  | 触发该用例的主参与者与次参与者 |
| 前置条件 | 用例开始前系统必须满足的状态  |
| 主流程  | 正常路径的编号步骤序列     |
| 备选流程 | 异常或分支路径的处理      |
| 后置条件 | 用例结束后系统的状态与产出   |

## 第三章 结构化分析与设计

结构化方法从「数据流」与「模块」两个视角刻画系统：用数据流图（DFD）描述数据如何流动与变换，用数据字典精确定义数据，再用「高内聚、低耦合」的模块化原则指导设计。本章建立「DFD → 数据字典 → 模块化」的分析与设计主线。

### 3.1 数据流图（DFD）

数据流图（Data Flow Diagram, DFD）从数据流动的角度描述系统「做什么」：只关心数据及其变换，不关心控制时序与实现细节。

#### 3.1.1 四种基本元素

表 3.1: DFD 的四种基本元素

| 元素   | 图形     | 说明                               |
|------|--------|----------------------------------|
| 外部实体 | 矩形     | 系统边界外的参与者（人 / 组织 / 外部系统），是数据的源或宿 |
| 加工   | 圆 / 椭圆 | 对数据执行变换的处理单元，命名用「动词 + 名词」        |
| 数据存储 | 开口矩形   | 数据的静态仓库（文件 / 表 / 库），只能由加工读写      |
| 数据流  | 带箭头的线  | 沿箭头方向流动的数据，命名用名词，不代表控制或时序        |

#### 3.1.2 分层 DFD



图 3.1: 0 层 DFD：上下文图

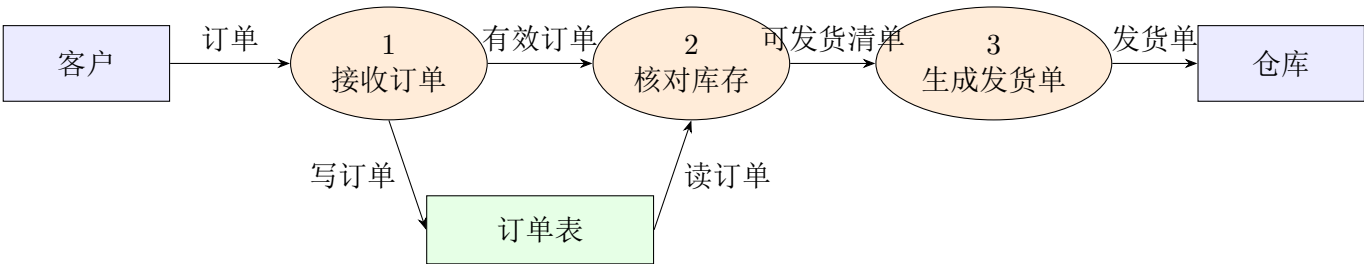


图 3.2: 1 层 DFD：订单处理系统分解

#### 3.1.3 平衡原则

对加工逐层分解，可得到 0 层、1 层、2 层 ... DFD：

- 0 层（上下文图）：把整个系统看作一个加工，只画系统与外部实体之间的数据交换。
- 1 层：把 0 层的加工分解成若干子加工，并画出与之相关的数据存储。
- 平衡（父子平衡）：子图边界上的输入 / 输出数据流，必须与父图中该加工的输入 / 输出一一对应——分解时数据既不凭空产生，也不凭空消失。
- 编号：加工编号体现层次，如 1 分解为 1.1、1.2；数据流与数据存储也都要命名。

## 3.2 数据字典

数据字典（Data Dictionary）为 DFD 中的每个数据流、数据存储与数据项给出精确定义，是 DFD 的「词条说明」。常用下列记号：

| 表 3.2: 数据字典常用记号 |              |                       |
|-----------------|--------------|-----------------------|
| 记号              | 含义           | 例                     |
| =               | 被定义项 = 其组成   | 订单 = 订单号 + 客户 + {订单项} |
| +               | 顺序连接（逻辑与）    | 客户 = 姓名 + 地址 + 电话     |
| { }             | 重复（0 次或多次）   | {订单项}                 |
| [ ]             | 从中选择一项       | 性别 = [男   女]          |
|                 | 在 [ ] 内分隔备选项 | 男   女                 |
| ( )             | 可选（可有可无）     | 电话 = 座机 + (手机)        |

例如订单的数据字典：

订单 = 订单号 + 客户 + {订单项} + 下单日期  
订单项 = 商品号 + 数量 + 单价  
客户 = 客户号 + 姓名 + (联系电话)

数据字典与 DFD 必须保持一致：图里出现的每个名字，都应在字典中有精确的定义。

## 3.3 模块化设计：耦合与内聚

模块化把系统划分为相对独立、单一职责的模块。衡量划分优劣的两个指标是**耦合**（模块之间联系的紧密程度）与**内聚**（模块内部各成分结合的紧密程度）。设计目标是**高内聚、低耦合**。

### 3.3.1 耦合（Coupling）

耦合由低到高依次为：数据耦合、标记耦合、控制耦合、公共耦合、内容耦合（更完整地还包含非直接耦合与外部耦合）。耦合越低越好。

| 表 3.3: 耦合类型（从低到高） |        |                          |
|-------------------|--------|--------------------------|
| 类型                | 传递内容   | 说明                       |
| 非直接耦合             | 无      | 模块之间无直接联系，最好             |
| 数据耦合              | 简单数据   | 通过参数传值（如传一个数），最常用的良好耦合   |
| 标记耦合              | 数据结构   | 传整个记录 / 结构，对方只用其中一部分     |
| 控制耦合              | 控制信息   | 传开关 / 标志以控制对方逻辑，增加理解难度   |
| 外部耦合              | 外部约束   | 都受同一外部环境 / 数据格式约束        |
| 公共耦合              | 公共数据区  | 多个模块共享全局变量，牵一发动全身        |
| 内容耦合              | 直接访问内部 | 一个模块直接访问另一模块内部数据 / 代码，最差 |

### 3.3.2 内聚（Cohesion）

内聚由高到低依次为：功能内聚、顺序内聚、通信内聚、过程内聚、时间内聚、逻辑内聚、偶然内聚。内聚越高越好。

| 表 3.4: 内聚类型（从高到低） |      |                         |
|-------------------|------|-------------------------|
| 类型                | 结合方式 | 说明                      |
| 功能内聚              | 单一功能 | 所有成分共同完成一个明确功能，最好       |
| 顺序内聚              | 数据顺序 | 上一步的输出是下一步的输入，数据顺序流动    |
| 通信内聚              | 同一数据 | 各成分操作同一输入数据或产生同一输出数据    |
| 过程内聚              | 执行顺序 | 各成分按固定流程顺序执行，但数据联系较弱    |
| 时间内聚              | 同一时段 | 各成分因需在同一时间段内执行而聚合（如初始化） |
| 逻辑内聚              | 逻辑相似 | 各成分逻辑上同类，由参数选择执行哪一部分    |
| 偶然内聚              | 无关系  | 各成分之间无明显关系，最差           |

**结论：**理想的模块划分是「高内聚、低耦合」——模块内部职责单一且各成分密切相关，模块之间只通过简单数据进行必要的通信。理想组合是「功能内聚 + 数据耦合」，这样模块更易理解、测试、复用与维护。

## 第四章 面向对象与 UML

面向对象方法以「对象」为核心组织软件，UML 用一组标准图从**结构**（类图、用例图）与**行为**（时序图、状态图、活动图）两个视角刻画系统。本章建立「类-关系-交互-状态」的建模主线。

## 4.1 类图

类图是 UML 中最核心的**结构图**，描述系统的静态结构：类、接口以及它们之间的关系。一个类通常画成**三格矩形**：类名、属性、方法。

### 4.1.1 类与成员

表 4.1: 成员的可见性修饰符

| 符号 | 可见性            | 含义       | 示例                  |
|----|----------------|----------|---------------------|
| +  | public（公有）     | 任何类都可访问  | + getName(): String |
| -  | private（私有）    | 仅本类可访问   | - balance: Money    |
| #  | protected（受保护） | 本类及子类可访问 | # id: String        |
| ~  | package（包）     | 同包内可访问   | ~cache: Map         |

成员书写的通用格式为「**可见性名称: 类型 [多重性] = 默认值**」；方法为「**可见性名称 (参数): 返回类型**」。静态成员加**下划线**，抽象成员用**斜体**（或类名用 {abstract} 标注）。

4.1.2 类之间的关系

表 4.2: 六种类关系的符号与语义

| 关系                  | 符号                 | 语义                      | 耦合 |
|---------------------|--------------------|-------------------------|----|
| 关联 (Association)    | 实线 (可带方向箭头)        | 类之间的一般连接, 如「教师—课程」      | 弱  |
| 聚合 (Aggregation)    | 空心菱形 + 实线 (菱形在整体端) | 整体—部分, 部分可独立存在 (has-a)  | 中  |
| 组合 (Composition)    | 实心菱形 + 实线 (菱形在整体端) | 整体—部分, 部分随整体消亡 (owns-a) | 强  |
| 泛化 (Generalization) | 空心三角 + 实线 (三角指向父类) | 继承关系 is-a, 子类复用父类       | 强  |
| 依赖 (Dependency)     | 虚线 + 开放箭头          | 临时使用, 如参数、局部变量、返回值      | 弱  |
| 实现 (Realization)    | 虚线 + 空心三角 (指向接口)   | 类实现接口所定义的契约             | 中  |

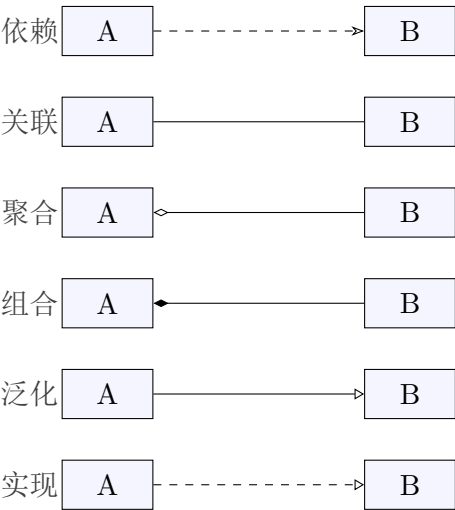


图 4.1: 六种类关系的图形符号 (关联/聚合/组合以 A 为整体端; 泛化/实现以 B 为父类或接口端)

4.1.3 多重性

多重性 (multiplicity) 标注在关联线的两端, 表示一个对象在给定时刻可与多少个对端对象相关联。

表 4.3: 常用的多重性标注

| 标注   | 读法   | 示例           |
|------|------|--------------|
| 1    | 恰好一个 | 一张订单只属于一个客户  |
| 0..1 | 零或一个 | 可选引用         |
| *    | 零或多个 | 一个客户可有任意多张订单 |
| 1..* | 至少一个 | 订单至少包含一个订单项  |
| n..m | 区间   | 例如 2..4 个玩家  |

#### 4.1.4 类图示例

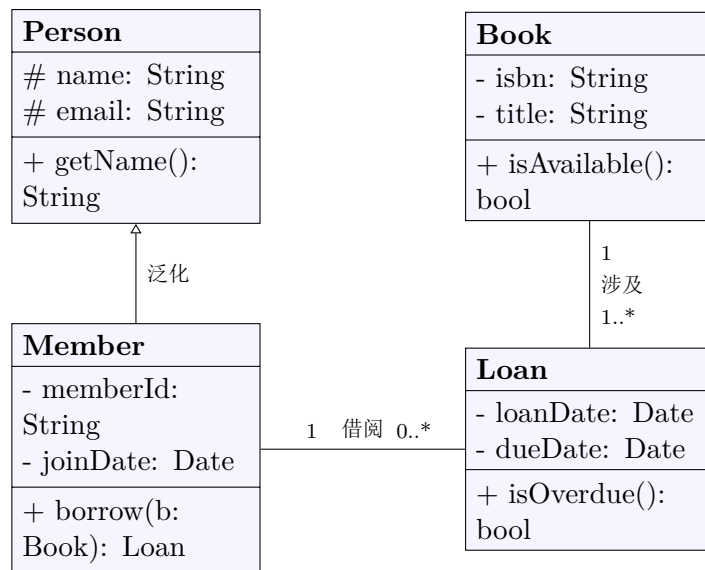


图 4.2: 类图示例：图书馆借阅系统（Member 继承 Person，借阅记录 Loan 关联 Member 与 Book）

- 泛化：Member 是 Person 的子类，继承 name、email 等成员。
- 关联 + 多重性：一个 Member 可产生 0..\* 条 Loan；每条 Loan 恰好对应 1 个 Member。
- 属性/方法的类型：String、Date、bool 为数据类型，不参与对象间关联。

## 4.2 用例图

用例图从外部视角描述系统「为谁提供什么服务」，是需求分析阶段与用户沟通的主要工具。

### 4.2.1 要素与关系

表 4.4: 用例图的主要要素

| 要素                  | 图形                    | 说明            |
|---------------------|-----------------------|---------------|
| 参与者 (Actor)         | 火柴人                   | 系统外部的用户或外部系统  |
| 用例 (Use Case)       | 椭圆                    | 系统提供的一项完整功能   |
| 系统边界 (Boundary)     | 矩形框                   | 框内是用例，框外是参与者  |
| 关联 (Association)    | 实线                    | 参与者与用例的通信     |
| 包含 (include)        | 虚线箭头 + <i>include</i> | 被包含用例是必选的公共步骤 |
| 扩展 (extend)         | 虚线箭头 + <i>extend</i>  | 扩展用例在特定条件下才插入 |
| 泛化 (Generalization) | 空心三角                  | 参与者或用例的继承     |

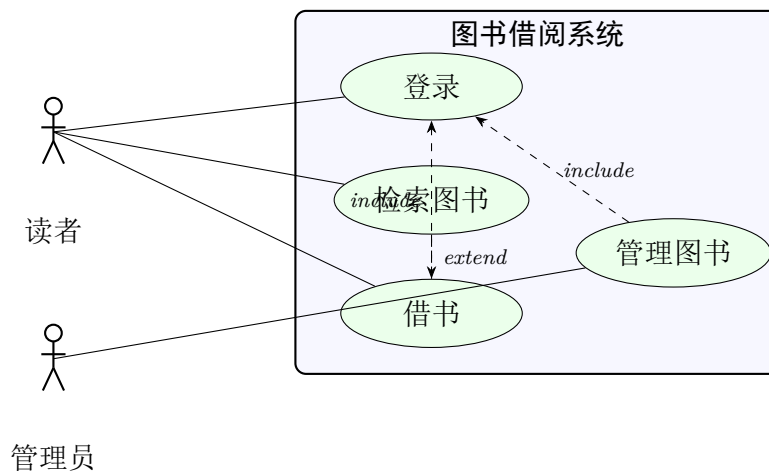


图 4.3: 用例图示例：读者与管理员使用图书借阅系统

**include 与 extend 的区别：**include 是把多个用例共有的步骤抽成公用例（**必然执行**）；extend 是给基础用例附加可选行为（**条件触发**）。箭头方向也不同：include 由基础用例指向被包含用例，extend 由扩展用例指向基础用例。

## 4.3 时序图

时序图（顺序图）是**交互图**，强调消息在时间上的先后顺序，用于描述一个用例内部对象之间的协作。

### 4.3.1 要素

表 4.5: 时序图的主要要素

| 要素                  | 图形        | 说明           |
|---------------------|-----------|--------------|
| 生命线 (Lifeline)      | 参与者下方的虚线  | 对象在时间轴上的存在   |
| 激活条 (Activation)    | 生命线上的窄矩形  | 对象正在执行（控制焦点） |
| 同步消息 (Synchronous)  | 实线 + 实心箭头 | 调用后等待返回，阻塞   |
| 异步消息 (Asynchronous) | 实线 + 开放箭头 | 发送后不等待，继续执行  |
| 返回消息 (Return)       | 虚线 + 开放箭头 | 把结果返回给调用者    |
| 自消息 (Self)          | 折返箭头      | 对象调用自身方法     |

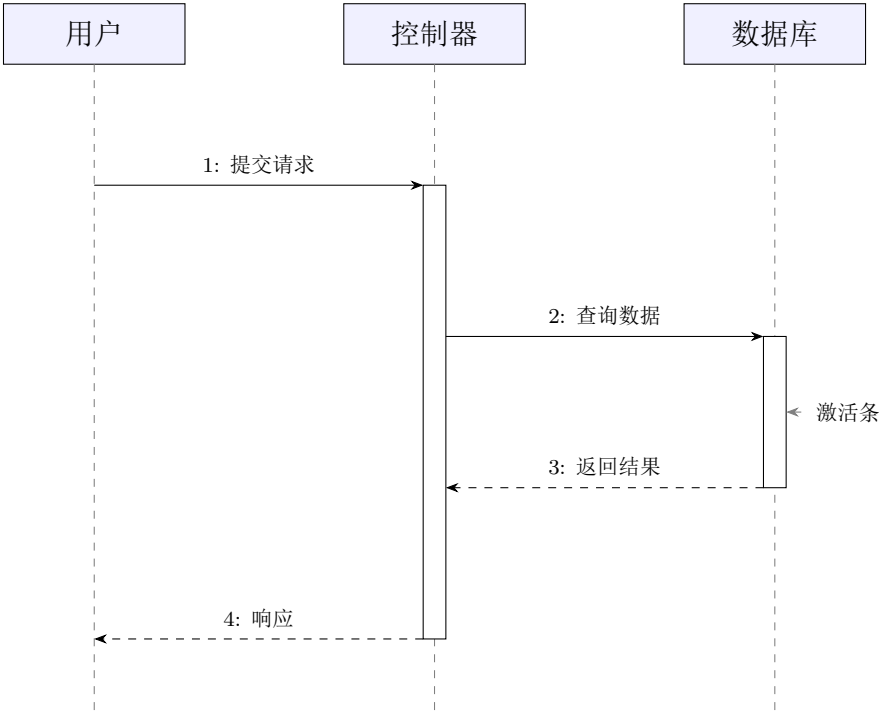


图 4.4: 时序图示例：一次查询交互（实线为同步消息，虚线为返回消息）

**同步与异步：**同步消息的发送者在收到返回前被阻塞，通常配套一个返回消息；异步消息发送后立即继续，不等待返回，常用于事件通知。激活条的高度反映对象执行该操作的时长。

## 4.4 状态图与活动图

### 4.4.1 状态图

状态图描述单个对象在其生命周期内状态的变化，转移的通用语法为：

事件 [守卫] / 动作

(4.1)

即「发生某事件，在守卫条件成立时，执行某动作并迁移到目标状态」。

表 4.6: 状态图的主要要素

| 要素              | 图形        | 说明          |
|-----------------|-----------|-------------|
| 状态 (State)      | 圆角矩形      | 对象所处的条件或状况  |
| 初始状态            | 实心圆       | 唯一的入口       |
| 终止状态            | 牛眼（圆内实心圆） | 生命周期结束      |
| 转移 (Transition) | 带箭头实线     | 状态之间的迁移     |
| 事件 (Event)      | 转移上的触发名   | 触发转移的信号     |
| 守卫 (Guard)      | [条件]      | 转移能否发生的布尔条件 |
| 动作 (Action)     | /动作       | 转移发生时执行的行为  |

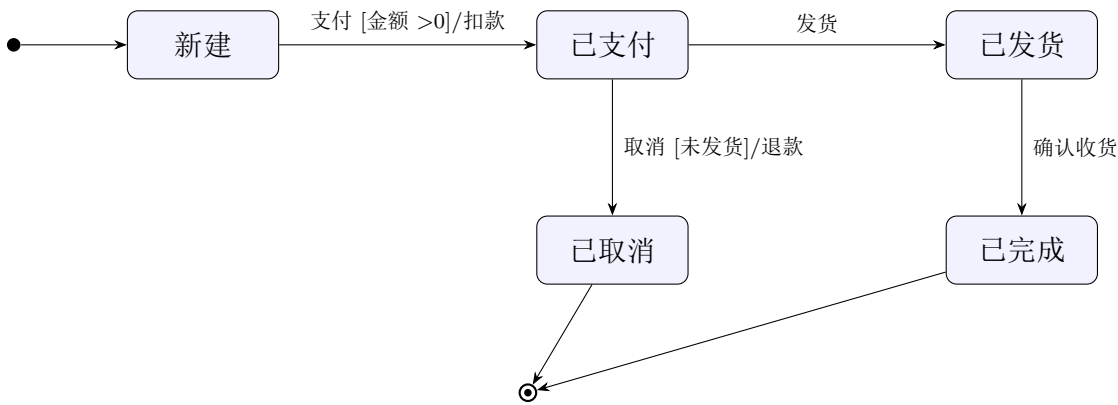


图 4.5: 订单状态图：事件 [守卫]/动作

4.4.2 活动图

活动图描述工作流或算法流程，与流程图相近，但支持并发、泳道等语义。

表 4.7: 活动图的主要要素

| 要素                | 图形   | 说明        |
|-------------------|------|-----------|
| 开始 (Initial)      | 实心圆  | 流程入口      |
| 活动 (Action)       | 圆角矩形 | 一个可执行的步骤  |
| 判断 (Decision)     | 菱形   | 依据条件选择分支  |
| 合并 (Merge)        | 菱形   | 多条分支汇合    |
| 分叉/汇合 (Fork/Join) | 粗黑线  | 并发流的开始与同步 |
| 泳道 (Swimlane)     | 分区   | 按负责对象划分活动 |
| 结束 (Final)        | 牛眼   | 流程终止      |

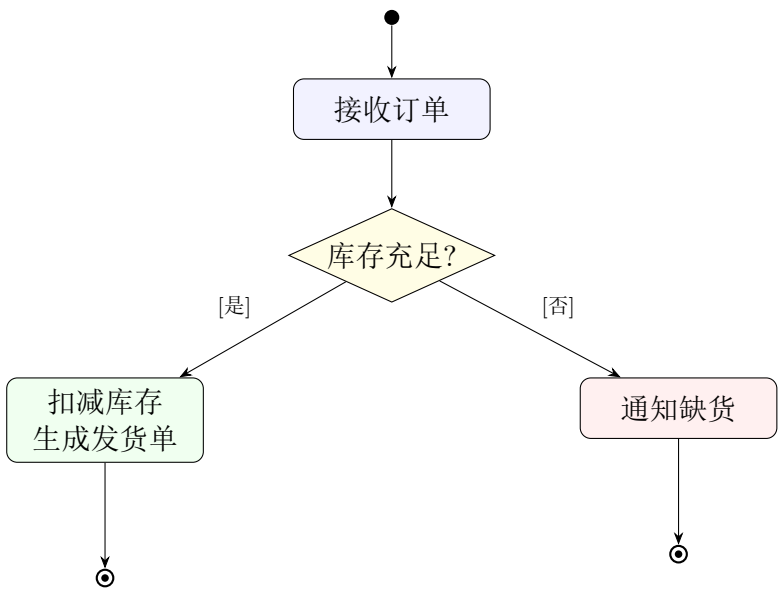


图 4.6: 活动图示例：订单处理流程（含判断分支）

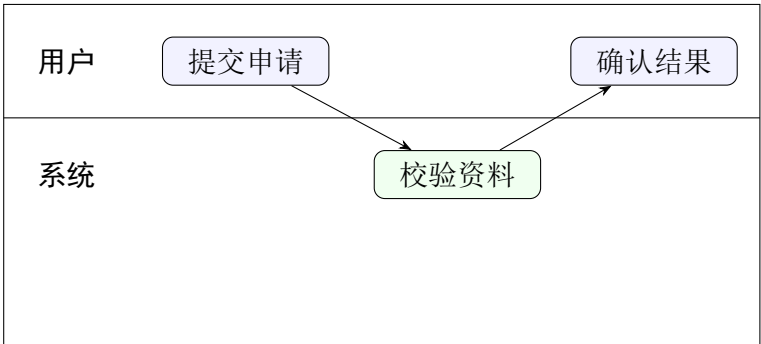


图 4.7: 泳道活动图：按「用户/系统」分区，箭头跨泳道表示职责交接

**状态图 vs. 活动图：**状态图刻画一个对象因事件驱动的状态变迁（关注「对象处于什么状态」）；活动图刻画一个流程中动作与控制的流转（关注「下一步做什么」）。二者都可建模行为，但抽象对象与关注点不同。

## 第五章 数据建模与 ER

数据建模把现实世界抽象为「实体-联系-属性」，再用 ER 图与关系模式两种语言描述同一份语义。本章从 ER 的基本构件出发，给出到关系模式的转换规则，并用函数依赖与范式刻画「好」的模式。

# 5.1 实体、属性与键

- **实体 (Entity):** 客观存在且可相互区分的事物；同类实体的集合称为**实体集**。
- **属性 (Attribute):** 实体具有的特征。可分为简单/复合、单值/多值、存储/派生属性。
- **键 (Key):** 能唯一标识实体的属性或属性组，是实体集与其属性的核心约束。
- **弱实体 (Weak Entity):** 自身没有完整主键，必须依附于某个**强实体**（标识实体）而存在；它的**部分键**只在本实体内唯一。

表 5.1: 键的分类

| 键                   | 含义                  | 示例          |
|---------------------|---------------------|-------------|
| 超键 (Superkey)       | 能唯一标识元组的属性组（可含冗余属性） | {学号, 姓名}    |
| 候选键 (Candidate Key) | 最小的超键，去掉任一属性都不能唯一标识 | {学号}、{身份证号} |
| 主键 (Primary Key)    | 从候选键中指定的一个，非空、唯一    | 学号          |
| 外键 (Foreign Key)    | 引用另一关系主键的属性（组），可为空  | 选课. 学号      |
| 部分键 (Partial Key)   | 弱实体内用于区分同类实体的属性     | 订单项的「行号」    |

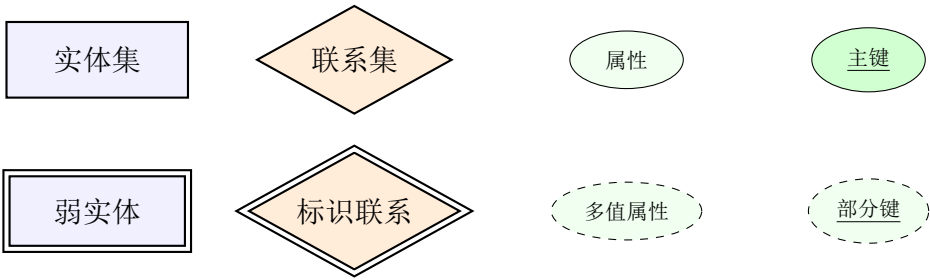


图 5.1: ER 图的基本符号：矩形 = 实体集，菱形 = 联系集，椭圆 = 属性，下划线 = 主键，双线 = 弱实体/标识联系，虚线 = 多值或部分键

# 5.2 联系与基数

联系 (Relationship) 刻画实体之间的语义关联。其**基数**（一对一、一对多、多对多）由参与实体集的映射基数决定；**参与约束**则规定实体是否必须参与联系。

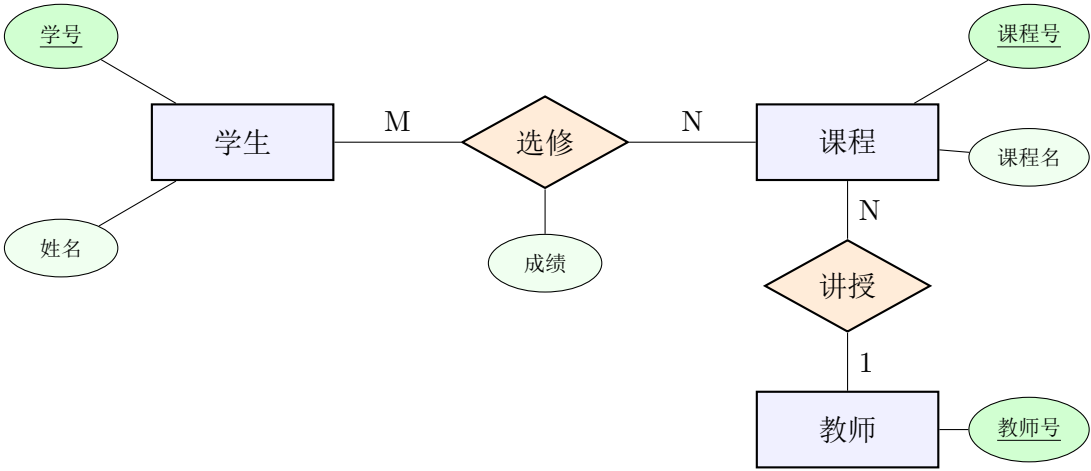


图 5.2: 一个 ER 示例：学生与课程为  $M:N$  的「选修」（含属性「成绩」），教师与课程为  $1:N$  的「讲授」

表 5.2: 基数与到关系模式的转换

| 基数  | 语义         | 转换规则                      |
|-----|------------|---------------------------|
| 1:1 | 两侧各至多一个对方  | 外键放在任一侧（加 UNIQUE），或合并为一张表 |
| 1:N | 一侧一个、另一侧多个 | 把外键放在 N 端，指向 1 端的主键       |
| M:N | 两侧均可多个     | 新建中间表，主键为双方主键的组合，联系属性也放入  |

**参与约束：**若某实体集中每个实体都必须参与联系，称**全参与**（图中用双线，也称存在依赖）；否则为**部分参与**（单线）。例如「每个学生必须选修至少一门课」即学生侧全参与。

**属性化联系：**联系自身也可带属性。例如「选修」带有「成绩」。当联系为  $M:N$  时，这些属性恰好随中间表保存；若为  $1:N$ ，则应放到  $N$  端的关系中。

### 5.3 ER 到关系模式的转换

把 ER 图转换为关系模式（一组带主键、外键的表）遵循下列规则：

1. **强实体集** → 一张表，取其主键作为该表主键。
2. **弱实体集** → 一张表，主键 = 标识实体的主键 + 自身部分键。
3. **1:1 联系**：把一方主键作为外键放入另一方（唯一约束）。
4. **1:N 联系**：把 1 端主键作为外键放入  $N$  端。
5.  **$M:N$  联系**：新建**联系表**，主键为参与双方主键的组合，并加入联系属性。
6. **多值属性**：为其单独建表，主键 = 所属实体主键 + 该属性。

以图中的例子为例，转换结果为：

学生(学号, 姓名), 课程(课程号, 课程名),  
 教师(教师号), 选修(学号, 课程号, 成绩),  
 讲授(教师号, 课程号)

(5.1)

## 5.4 函数依赖与范式

**函数依赖** (Functional Dependency, FD)  $X \rightarrow Y$  表示：在任何合法关系中，只要两个元组在  $X$  上相等，它们在  $Y$  上也必相等。

$$\forall t_1, t_2 : t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y] \tag{5.2}$$

在候选键  $K$  下， $K \rightarrow$  所有属性；若  $X \rightarrow Y$  且  $Y \rightarrow Z$ ，则  $X \rightarrow Z$  (传递)。

表 5.3: 范式的定义与所消除的异常

| 范式   | 条件                                     | 消除的问题         |
|------|----------------------------------------|---------------|
| 1NF  | 所有属性都是原子的 (不可再分)                       | 多值属性、重复组      |
| 2NF  | 1NF，且每个非主属性完全依赖于候选键                    | 非主属性对候选键的部分依赖 |
| 3NF  | 2NF，且非主属性不传递依赖于候选键                     | 非主属性对候选键的传递依赖 |
| BCNF | 每个非平凡 $X \rightarrow A$ 的决定因素 $X$ 都是超键 | 主属性之间的依赖      |

$$3NF : \quad \forall X \rightarrow A \in F^+, \quad X \text{ 是超键} \vee A \text{ 是主属性} \tag{5.3}$$

$$BCNF : \quad \forall X \rightarrow A \in F^+, A \notin X, \quad X \text{ 是超键} \tag{5.4}$$

规范化通过**分解**逐步消除冗余与更新异常：1NF 拆分多值单元格，2NF 按完全依赖分解，3NF 消除传递依赖。BCNF 比 3NF 更严格，可能损失函数依赖保持性；反之 3NF 总能做到无损且保持依赖。

## 第六章 设计原则与模式

良好的设计在「可扩展」与「可维护」之间取得平衡：SOLID 原则指导类的职责划分，设计模式沉淀可复用的协作经验，软件架构决定系统的整体骨架。

## 6.1 SOLID 五原则

SOLID 是一组面向对象设计的指导思想，目标是让系统在需求变化时「改动局部、稳定整体」。

表 6.1: SOLID 五原则：含义与反例改进

| 原则         | 含义              | 反例 → 改进                              |
|------------|-----------------|--------------------------------------|
| 单一职责 (SRP) | 一个类只有一个引起变化的原因  | 上帝类 → 按职责拆分                          |
| 开闭 (OCP)   | 对扩展开放、对修改关闭     | 每加类型就改 <code>switch</code> → 引入多态/策略 |
| 里氏替换 (LSP) | 子类型可替换父类型而不破坏契约 | 子类抛异常、加强前置条件 → 保持契约                  |
| 接口隔离 (ISP) | 客户端不应依赖不需要的接口   | 胖接口 → 拆成小而专的接口                       |
| 依赖倒置 (DIP) | 高层与低层都依赖抽象      | 高层 <code>new</code> 具体实现 → 依赖接口注入    |

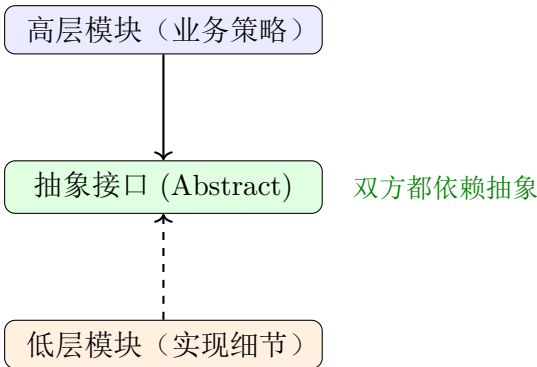


图 6.1: 依赖倒置：高层与低层都依赖抽象，抽象不依赖细节

依赖倒置把「高层直接依赖低层」翻转为「双方依赖抽象」，从而使业务策略不受具体实现替换的影响，也便于替换为测试替身。

## 6.2 设计模式

### 6.2.1 模式分类

表 6.2: 设计模式三大分类

| 类型               | 关注点            | 代表模式                |
|------------------|----------------|---------------------|
| 创建型 (Creational) | 对象创建机制，解耦创建与使用 | 单例、工厂方法、抽象工厂、建造者、原型 |
| 结构型 (Structural) | 类与对象的组合，组装更大结构 | 适配器、装饰器、代理、外观、组合    |
| 行为型 (Behavioral) | 对象间职责分配与通信     | 观察者、策略、命令、状态、模板方法   |

6.2.2 代表模式

表 6.3: 代表设计模式的意图与适用场景

| 模式                    | 意图                    | 适用               |
|-----------------------|-----------------------|------------------|
| 单例 (Singleton)        | 保证一个类仅有一个实例并提供全局访问点   | 全局配置、连接池、日志器     |
| 工厂方法 (Factory Method) | 定义创建对象的接口，由子类决定实例化哪个类 | 框架扩展点、解耦创建与使用    |
| 观察者 (Observer)        | 一对多依赖，主题变化时自动通知所有观察者  | 事件通知、GUI 事件、消息订阅 |
| 策略 (Strategy)         | 封装可互换的算法，使其可独立变化      | 多种计费/排序/压缩算法切换   |
| 装饰器 (Decorator)       | 动态给对象添加职责，比继承更灵活      | 流式 I/O、动态叠加功能    |

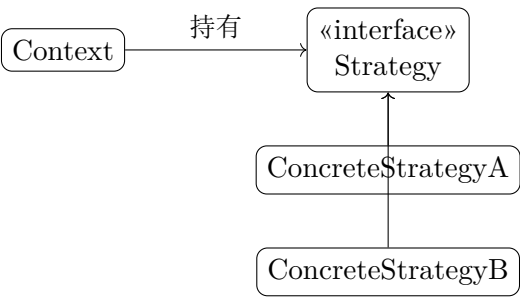


图 6.2: 策略模式：上下文持有抽象策略，运行时替换具体算法

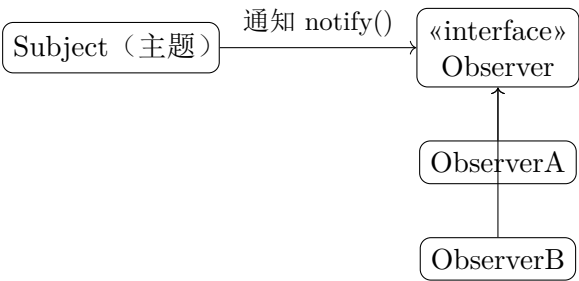


图 6.3: 观察者模式：主题在状态变化时遍历通知观察者列表

### 6.3 软件架构

表 6.4: 常见软件架构对比

| 架构                     | 特点                                 | 优点 / 代价                                         |
|------------------------|------------------------------------|-------------------------------------------------|
| 分层 (Layered)<br>MVC    | 上层调用下层，层间通过接口隔离模型、视图、控制器分离，视图随模型更新 | 结构清晰、易测试 / 调用穿透与性能损耗<br>UI 与业务解耦、多视图复用 / 控制器易膨胀 |
| 微服务<br>(Microservices) | 按业务边界拆分为独立部署的服务                    | 独立伸缩、技术异构、故障隔离 / 分布式与运维复杂                       |
| 事件驱动<br>(Event-driven) | 组件通过事件异步通信，发布-订阅                   | 松耦合、可扩展、异步削峰 / 最终一致、调试困难                        |

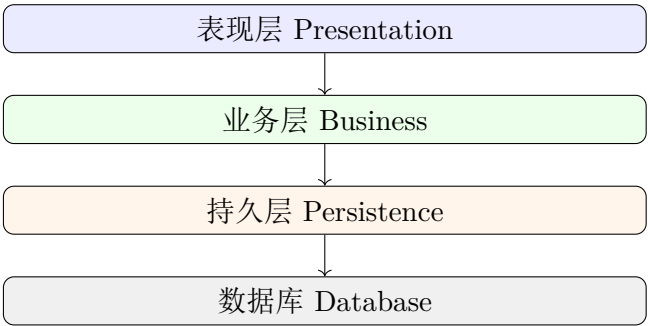


图 6.4: 分层架构：关注点自上而下逐层分离

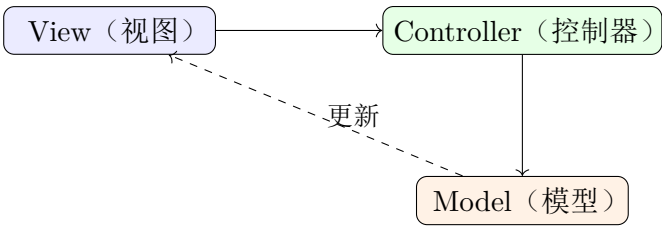


图 6.5: MVC：控制器处理输入、模型保存状态、视图订阅模型

架构选型的核心是权衡：分层适合关注点分离的常规业务系统；MVC 适合交互式 UI；微服务适合需要独立伸缩与团队自治的大型系统；事件驱动适合高并发、松耦合的集成场景。

## 第七章 软件测试

软件测试通过「执行程序并观察结果」来暴露缺陷，是质量保证的最后一道防线。本章覆盖测试层次与 V 模型、黑盒与白盒测试方法，以及测试用例设计与回归测试。

# 7.1 测试层次与 V 模型

软件测试按「由小到大、由内到外」分为四个层次：先保证单元正确，再保证组装正确，最后验证系统与业务目标。

- **单元测试**：验证单个函数或模块的内部逻辑与分支。
- **集成测试**：验证模块之间的接口、数据传递与协作。
- **系统测试**：在完整系统上验证功能需求与非功能需求。
- **验收测试**：由用户确认系统是否满足业务需求（ $\alpha$  测试、 $\beta$  测试）。

表 7.1: 测试层次的目标与执行者

| 层次   | 目标              | 执行者     |
|------|-----------------|---------|
| 单元测试 | 验证单个函数/模块的内部逻辑  | 开发人员    |
| 集成测试 | 验证模块间接口与数据传递    | 开发/测试人员 |
| 系统测试 | 验证完整系统的功能与非功能需求 | 独立测试团队  |
| 验收测试 | 用户确认是否满足业务需求    | 用户/客户   |

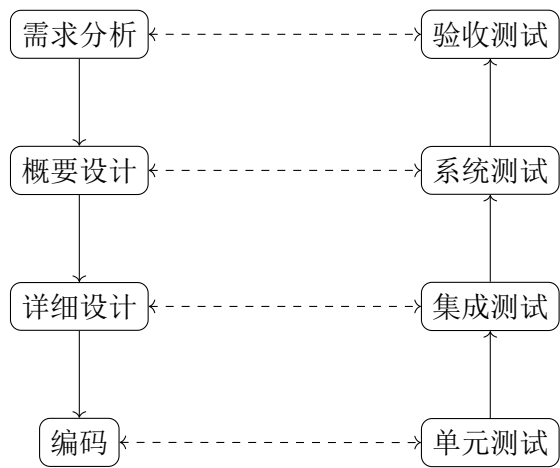


图 7.1: V 模型：左侧开发阶段与右侧测试阶段一一对应

V 模型把开发与测试对应起来：左边自顶向下逐层细化（需求 → 编码），右边自底向上逐层验证（单元 → 验收）。每一层的测试计划应在对应开发阶段就制定——需求阶段写验收/系统测试计划，设计阶段写集成测试计划，编码阶段写单元测试计划，从而尽早发现缺陷，降低修复代价。

## 7.2 黑盒测试与白盒测试

### 7.2.1 黑盒测试

黑盒测试只关心「输入-输出」，不考虑程序内部结构，常用方法如下。

- **等价类划分**：把输入域划分为若干等价类，同类中输入任取一个即可代表整个类。分为有效等价类（合法输入）与无效等价类（非法输入）。
- **边界值分析**：缺陷大多藏在边界，取  $\text{min} - 1$ ,  $\text{min}$ ,  $\text{min} + 1$ ,  $\text{max} - 1$ ,  $\text{max}$ ,  $\text{max} + 1$  设计用例。
- **因果图**：把「原因（输入条件）」与「结果（输出动作）」用图形表达，再转化为判定表来设计测试用例，处理输入组合与约束。

表 7.2: 边界值分析示例（输入范围  $[1, 100]$ ）

| 边界点              | 取值  | 预期结果     |
|------------------|-----|----------|
| $\text{min} - 1$ | 0   | 无效（低于下界） |
| $\text{min}$     | 1   | 有效       |
| $\text{min} + 1$ | 2   | 有效       |
| $\text{max} - 1$ | 99  | 有效       |
| $\text{max}$     | 100 | 有效       |
| $\text{max} + 1$ | 101 | 无效（高于上界） |

### 7.2.2 白盒测试

白盒测试依据程序内部逻辑设计用例，常见覆盖准则由弱到强为：语句覆盖、判定覆盖、条件覆盖、路径覆盖。

- **语句覆盖**：每条可执行语句至少执行一次。
- **判定覆盖（分支覆盖）**：每个判定的真、假分支至少各执行一次。
- **条件覆盖**：每个原子条件的真、假至少各取一次。
- **路径覆盖**：每条独立路径至少执行一次，最强但路径数可能指数增长。

```
1 int f(int a, int b) {
2     int x = 0;
3     if (a > 0 && b > 0) {    /* 判定 D, 含原子条件 a>0 与 b>0 */
4         x = a + b;
5     }
6     return x;
7 }
```

表 7.3: 覆盖准则强弱与示例最少用例数（短路求值）

| 覆盖准则 | 含义                       | 本例最少用例数                     |
|------|--------------------------|-----------------------------|
| 语句覆盖 | 每条可执行语句至少一次              | 1（如 $a = 2, b = 2$ ）        |
| 判定覆盖 | 判定 $D$ 的真、假各一次           | 2（如 $(2, 2)$ 、 $(-1, 2)$ ）  |
| 条件覆盖 | $a > 0$ 、 $b > 0$ 的真假各一次 | 2（如 $(2, 2)$ 、 $(-1, -1)$ ） |
| 路径覆盖 | 每条独立路径至少一次               | 3（短路三条路径）                   |

强弱关系：满足判定覆盖一定满足语句覆盖（判定覆盖  $\supseteq$  语句覆盖）；条件覆盖不一定满足判定覆盖，需「判定/条件覆盖」同时保证；路径覆盖最强，但独立路径数随分支爆炸，工程上常用基于圈复杂度  $V(G) = E - N + 2$  的基本路径覆盖加以控制。

### 7.3 测试用例设计与回归测试

一个好的测试用例应包含四要素：前置条件、输入数据、操作步骤、预期输出，并且要能唯一判定「通过/失败」。

- 设计顺序通常为：先划分等价类  $\rightarrow$  补充边界值  $\rightarrow$  用因果图/判定表处理组合  $\rightarrow$  用白盒准则补齐未覆盖分支。
- 回归测试：修改代码后重新运行原有测试，确认没有引入新的缺陷或破坏既有功能。

表 7.4: 回归测试的常见策略

| 策略    | 说明              |
|-------|-----------------|
| 全量回归  | 重跑全部用例，最安全但代价最高 |
| 选择回归  | 只重跑受影响模块及其依赖的用例 |
| 优先级回归 | 先跑高风险、高优先级用例    |

## 第八章 项目管理与质量

项目管理回答四个问题：要多久、花多少、有多少风险、质量如何度量。本章沿「估算-进度-风险-质量」四步展开，把成本模型、关键路径法、风险应对与质量度量落到可计算的公式与表格上。

## 8.1 规模与成本估算

### 8.1.1 代码行 LOC 与功能点 FP

代码行 (LOC, Lines of Code) 是最直观的规模度量，通常以千行 **KLOC** 为单位。它易于统计，但依赖语言与编程风格，且在设计完成前难以准确预估。

功能点 (FP, Function Points) 从「用户可见的功能」出发，与语言无关。先按复杂度给五类功能元素加权，得到未调整功能点 *UFP*：

| 表 8.1: 功能点五类元素及复杂度权重 |    |    |    |
|----------------------|----|----|----|
| 功能元素                 | 简单 | 平均 | 复杂 |
| 外部输入 EI              | 3  | 4  | 6  |
| 外部输出 EO              | 4  | 5  | 7  |
| 外部查询 EQ              | 3  | 4  | 6  |
| 内部逻辑文件 ILF           | 7  | 10 | 15 |
| 外部接口文件 EIF           | 5  | 7  | 10 |

再用 14 项通用系统特征（数据通信、性能、重用性等）的复杂度  $F_i \in [0, 5]$  做调整：

$$FP = UFP \times \left( 0.65 + 0.01 \sum_{i=1}^{14} F_i \right) \tag{8.1}$$

其中调整因子取值范围为 0.65 ~ 1.35。功能点可在设计前估算，再换算成 KLOC 以套用成本模型。

### 8.1.2 COCOMO 模型

COCOMO (Constructive Cost Model) 把工作量表示为规模的幂函数。基本模型按项目类型取参数  $a, b, c, d$ ：

| 表 8.2: COCOMO 基本模型参数 |     |      |             |
|----------------------|-----|------|-------------|
| 项目类型                 | $a$ | $b$  | 说明          |
| 组织型 (Organic)        | 2.4 | 1.05 | 小型、经验丰富、约束少 |
| 半独立型 (Semi-detached) | 3.0 | 1.12 | 中等规模、混合经验   |
| 嵌入式 (Embedded)       | 3.6 | 1.20 | 硬约束、复杂接口    |

组织型对应的基本公式为

$$E = 2.4 \times (KLOC)^{1.05} \quad (\text{人月, person-months}) \tag{8.2}$$

$$D = 2.5 \times E^{0.38} \quad (\text{开发月数, months}) \tag{8.3}$$

$$\text{团队规模} = \frac{E}{D} \quad (\text{人}) \tag{8.4}$$

例如  $KLOC = 33$ （组织型）时： $E = 2.4 \times 33^{1.05} \approx 94.3$  人月， $D = 2.5 \times 94.3^{0.38} \approx 14.1$  月，团队规模约 6.7 人。可见工作量随规模**超线性**增长——把规模翻倍，工作量增长超过一倍，这正是「人月神话」的量化根源。

表 8.3: 常用成本估算方法对比

| 方法      | 优点         | 局限          |
|---------|------------|-------------|
| 代码行 LOC | 直观、易统计     | 语言相关、需设计后   |
| 功能点 FP  | 与语言无关、早期可用 | 主观、需经验校准    |
| COCOMO  | 公式化、可调参    | 依赖历史数据与规模估计 |

## 8.2 进度计划

### 8.2.1 甘特图

甘特图 (Gantt Chart) 用横条表示各活动的起止时间，直观展示进度与并行关系，是沟通与跟踪的主力工具。下面是本章示例项目的甘特图（时间单位统一，关键活动以深色标出）：

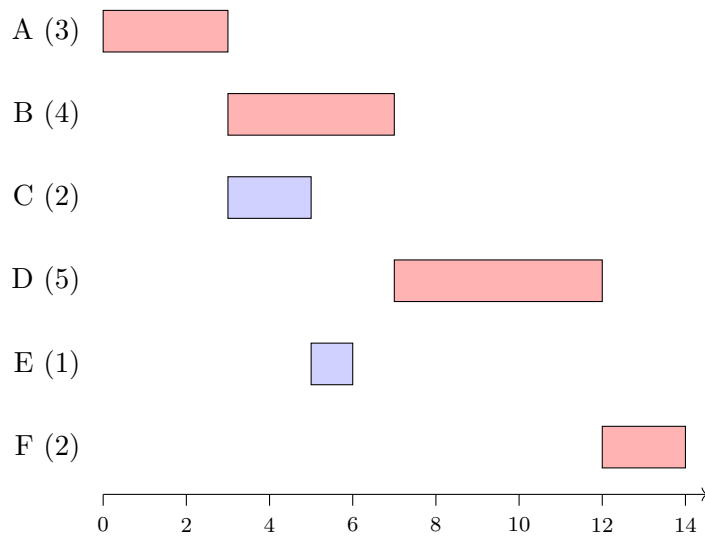


图 8.1: 项目甘特图（深色为关键活动，括号内为工期）

### 8.2.2 PERT 与关键路径法 CPM

关键路径法 (CPM) 把项目建成活动在网络 (AON)：节点是活动，箭头表示先后约束。对每个活动计算四个时间：

- 最早开始  $ES$ 、最早完成  $EF = ES + \text{工期}$ ：沿网络正向推进， $ES = \max(\text{所有前置的 } EF)$ 。
- 最晚完成  $LF$ 、最晚开始  $LS = LF - \text{工期}$ ：沿网络反向推进， $LF = \min(\text{所有后继的 } LS)$ ，终点  $LF$  取项目总工期。

总浮动时间 (Total Float) 为

$$TF = LS - ES = LF - EF \quad (8.5)$$

$TF = 0$  的活动组成关键路径，其长度等于项目最短工期；关键活动一旦延误，整个项目立即延误。示例项目活动与计算结果如下：

| 表 8.4: 示例项目活动表与 CPM 计算结果 |    |      |    |    |    |    |    |
|--------------------------|----|------|----|----|----|----|----|
| 活动                       | 工期 | 前置   | ES | EF | LS | LF | TF |
| A                        | 3  | —    | 0  | 3  | 0  | 3  | 0  |
| B                        | 4  | A    | 3  | 7  | 3  | 7  | 0  |
| C                        | 2  | A    | 3  | 5  | 9  | 11 | 6  |
| D                        | 5  | B    | 7  | 12 | 7  | 12 | 0  |
| E                        | 1  | C    | 5  | 6  | 11 | 12 | 6  |
| F                        | 2  | D, E | 12 | 14 | 12 | 14 | 0  |

关键路径为  $A \rightarrow B \rightarrow D \rightarrow F$ ，工期  $3 + 4 + 5 + 2 = 14$ 。活动 C、E 各有 6 个时间单位的浮动，可在不影响总工期的前提下适当推迟或调配资源。

## 8.3 风险管理

### 8.3.1 风险识别与评估

风险是「尚未发生、但一旦发生会危害项目」的潜在事件。管理流程为：识别 → 分析（概率 × 影响）→ 制定应对 → 监控。风险值用「发生概率」与「影响程度」的乘积度量：

$$R = P(\text{风险}) \times I(\text{影响}) \tag{8.6}$$

常见风险来源分四类：**技术**（技术不成熟、需求不稳）、**人员**（流失、技能不足）、**组织**（预算削减、变更失控）、**外部**（供应商、法规）。下表给出典型风险登记册：

| 表 8.5: 风险登记册示例（概率、影响取 1 ~ 5） |        |        |                  |                |
|------------------------------|--------|--------|------------------|----------------|
| 风险                           | 概率 $P$ | 影响 $I$ | $R = P \times I$ | 应对策略           |
| 需求蔓延                         | 4      | 4      | 16               | 缓解：变更控制、原型确认   |
| 关键技术不成熟                      | 3      | 4      | 12               | 规避：技术预研、留备选    |
| 核心人员流失                       | 2      | 5      | 10               | 缓解：文档化、结对、备份   |
| 第三方组件延误                      | 3      | 3      | 9                | 转移：合同条款、多供应商   |
| 预算削减                         | 3      | 5      | 15               | 缓解/接受：优先级排序与储备 |

8.3.2 风险应对策略

针对高  $R$  值风险，需选择并落实应对策略：

- 规避 (Avoid)：改变计划以消除风险源，如采用成熟技术替代未验证方案。
- 转移 (Transfer)：把后果转给第三方，如外包、保险、固定价合同。
- 缓解 (Mitigate)：降低发生概率或影响，如增加评审、冗余设计、缓冲。
- 接受 (Accept)：风险影响小或应对成本过高时，预留应急储备并监控。

表 8.6: 概率-影响矩阵与处置建议

|     | 影响低 | 影响中   | 影响高   |
|-----|-----|-------|-------|
| 概率高 | 缓解  | 缓解/规避 | 规避/转移 |
| 概率中 | 接受  | 缓解    | 规避/转移 |
| 概率低 | 接受  | 接受    | 缓解/转移 |

风险管理的输出是可跟踪的**风险登记册与应急储备**：不仅登记风险，还要指定负责人、触发条件与应对动作，并随项目推进持续复审。

8.4 软件质量

8.4.1 ISO 质量模型

**ISO/IEC 9126**（后演化为 ISO/IEC 25010）从多个质量特性刻画软件质量，常归纳为「功能性、可靠性、易用性、效率、可维护性、可移植性」等：

表 8.7: ISO 质量特性

| 质量特性                   | 含义              |
|------------------------|-----------------|
| 功能性 (Functionality)    | 满足所声明与隐含需求的能力   |
| 可靠性 (Reliability)      | 在规定条件下维持规定性能的能力 |
| 易用性 (Usability)        | 被理解、学习、使用的难易程度  |
| 效率 (Efficiency)        | 时间与资源利用的效率      |
| 可维护性 (Maintainability) | 被修改、纠错、适应、完善的能力 |
| 可移植性 (Portability)     | 迁移到其他环境的能力      |

8.4.2 质量度量

质量特性需用可度量的指标落地。常用度量包括：

$$D = \frac{\text{缺陷总数}}{\text{KLOC}} \quad (\text{缺陷密度, defects/KLOC}) \tag{8.7}$$

$$MTBF = MTTF + MTTR \tag{8.8}$$

$$A = \frac{MTTF}{MTTF + MTTR} \quad (\text{可用度, Availability}) \tag{8.9}$$

其中 *MTTF* 为平均无故障时间，*MTTR* 为平均修复时间。由公式可见：提高可用度有两条路——降低故障率（增大 *MTTF*）或加快恢复（减小 *MTTR*）。工程上常在两者间权衡，例如高可用系统用冗余提高 *MTTF*、用自动故障转移压缩 *MTTR*。

表 8.8: 常用质量度量指标

| 指标           | 定义                        | 关注点          |
|--------------|---------------------------|--------------|
| 缺陷密度         | 缺陷数 / KLOC                | 开发与测试阶段的缺陷控制 |
| 可靠性          | 失效概率、 <i>MTTF</i>         | 长时间稳定运行能力    |
| <i>MTBF</i>  | <i>MTTF</i> + <i>MTTR</i> | 平均故障间隔，可维护系统 |
| 可用度 <i>A</i> | <i>MTTF</i> / <i>MTBF</i> | 服务可用的时间占比    |

度量本身不是目的：质量模型提供「看哪些维度」，度量提供「量多大」，而评审、测试与过程改进才是「把质量做出来」的手段。